R7023E On-Board Data Handling

OBSW Software Design Document

Space Segment: Platform OBC and Payload ICU

Oliver Witzel, Isaac Edwards, Petter Bäckman, Anton Damberg

October 31, 2026

Luleå University of Technology

Contents

1 Introduction **2**

2 Architecture **2**

3 Functional blocks **3**

4 Communication **4**

4.1 Links 5

4.2 Telecommand format 5

4.3 Telemetry 5

4.4 Telecommands 5

4.5 Payload Interface 6

5 Modes and onboard time **6**

5.1 Payload states 7

5.2 Onboard time 7

6 Cyclic execution and key sequences **8**

6.1 OBC main loop (one cycle, PF8) 8

6.2 Payload 8

6.3 TC acceptance (PF3) 8

6.4 Image take, end to end 8

6.5 Switch to SAFE 8

7 Traceability matrix **9**

1 Introduction

The Space Segment software is the Platform OBC (PF) and the Payload ICU (PL), written in Python. The OBC runs in the obc Docker container; the payload is a separate module (`payload.py`) inside the OBC process. Ground runs in a second container.

Scope. Platform OBC and Payload ICU software for the R7023E OBDH assignment. The Ground MCS is described only where it meets the Space Segment (the uplink/downlink interface).

Reference documents

- R7023E Assignment description (J. Eickhoff, 2026)
- OBDH Function Tree workbook (sheets FT_Ground, FT_Space, Mode_Transitions, TC_TM_List and Requirements R01 to R32)

Table 1: Design decisions

Topic	Decision
Language	Python, standard library plus `spacepackets`, socket, selectors, time, threading
Deployment	2 Docker containers (ground and obc) on one Docker network, started with docker compose. The payload is a Payload class in `payload.py`.
Links	One TCP link, ground -> obc:5000. The OBC and payload talk by function calls. The OBC calls `pl.execute(cmd)`, and the payload reports back through `report = OBC.execute`
Message format	a CCDSD Space Packet primary header from the spacepackets library, followed by an ASCII data field with fields separated by underscores.
Time base	Host wall clock; all containers set to `UTC standard` so Gnd and OBC show the same local time.
Standards	CCSDS primary header used (APID for target, sequence count, length). No PUS services.

2 Architecture

The system is two Docker containers on one network. Ground connects to the OBC over TCP, the payload is a module inside the OBC process and is reached only through the OBC. (Figure 1).

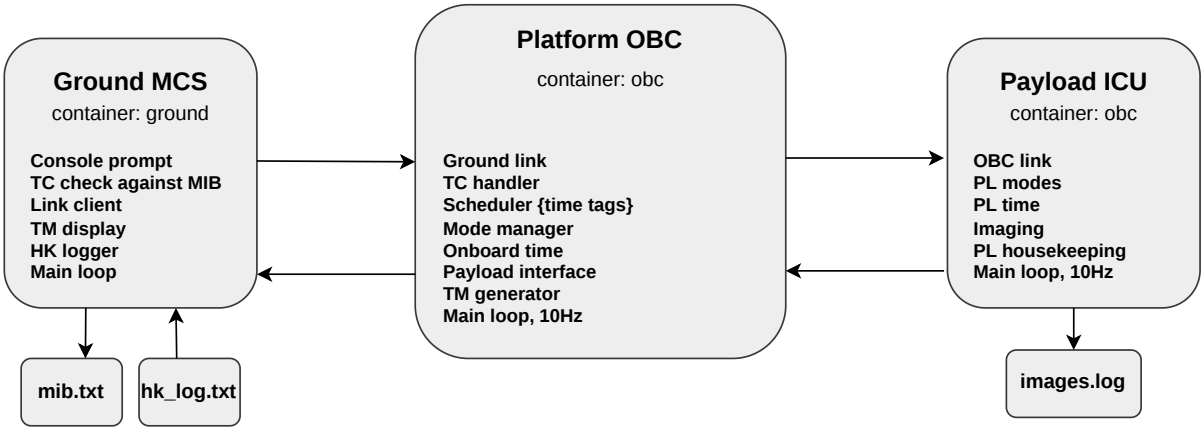


Figure 1: System architecture: two containers and one TCP link. Ground talks only to the OBC and only the OBC talks to the payload.

Ground reads the MIB at start and writes received HK to its logfile; the payload writes one line per image to its image log. Both programs read the shared `common/mib.txt`. Ground writes HK to `data/ground/HK_log.txt` and the payload writes image entries to `data/payload/Pl_log.txt`.

3 Functional blocks

Each block belows is one class or one group of methods. Ground, OBC, and Payload are classes, shared code is in `common.py`.

Table 2: Functional blocks

Block	Class / method	Responsibility
Ground (ground.py, container `ground`)		
Ground link	`Ground.connect`, `Ground.readTM`	TCP client to the OBC; receives TM packets
TC handling	`Ground.parse_cmd`, `Ground.sendTC`	Checks operator input, builds and sends TC packets
TM display + log	`Ground.print_log`	Prints TM; writes HK to the logfile with ground reception time
Ground main loop	`Ground.run`	Polls keyboard and socket every 0.2 s; ACK timeout 10 s
Platform OBC (obc.py, container `obc`)		
OBC ground link	`OBC.gnd_acc`, `OBC.gnd_read`	TCP server; accepts Ground, receives TC packets
TC handler	`OBC.on_tc`, `OBC.check_mode`	Decode, ID lookup, sequence check, mode checks, ACK/NACK
Executor	`OBC.execute`	Runs accepted TCs; forwards payload reports to Ground
Mode manager	`OBC.set_mode`, `OBC.check_transition`	SAFE / NADIR / TARGET with a 5 s TRANSITION
Onboard time	`OBC.now`	SCET until set, then OBT, in seconds
TM generator	`OBC.send_TM`, `ack`, `nack`, `event`, `obc_hk`, `pl_hk`	Builds TM packets; TM sequence count
OBC main loop	`OBC.run`	Polls the socket every 0.1 s; checks transitions
Payload ICU (payload.py, runs inside the OBC process)		
Payload control	`Payload.execute`, `turn_on`, `turn_off`	OFF / ON / IMAGING state
Imaging	`Payload.start_imaging`, `done_imaging`	5 s timer; writes the image log with OBT
Payload time	`Payload.sync_time`, `Payload.now`	Keeps the OBT received from the OBC
Payload HK	`Payload.hk`	Mode, images taken, last image time
Shared (common.py)		
Packet handling	`packer`, `Framer`	Builds CCSDS packets; reassembles complete packets from the TCP stream
Support	`SequenceCount`, `load_mib`, `s_to_HMS`, `HMS_to_s`	Sequence counters, MIB reading, time conversion

How the payload knows the OBT. When SET-OBT executes, the OBC calls `Payload.sync_time()` with the new OBT. The payload stores it together with its own monotonic clock reading and adds the elapsed seconds, so its image log uses onboard time.

4 Communication

Ground and OBC exchange CCSDS Space Packets over one TCP connection. The OBC and payload communicate by direct function calls.

4.1 Links

Table 3: TCP links

Link	Server	Client	Port	Carries
Space link	obc	ground	5000	TC up, TM down

Containers find each other by service name on the Docker network (ground connects to `obc:5000`, obc connects to `payload:5001`). Each side reads with a non-blocking socket (select with zero timeout) or a receiver thread inside its main loop, so no process stops while waiting.

4.2 Telecommand format

Headers:Type,APID,Sequence

```
"{command_id}_{args}_{timetag}"
```

4.3 Telemetry

Headers:Type,APID,Sequence

```
"{tm_type}_{obt}_{args}"
```

Table 4: Telemetry types

TM	Type ID	Args
ACK	1	cmd_id, tc_seq
NACK	2	cmd_id, tc_seq, reason
HK-PF	3	pl_power, tc_rx, tc_rej (on REQ-HK and cyclic)
HK-PL	4	mode, images, last_image (on REQ-PL-HK)
EVENT	5	event_id, text)

4.4 Telecommands

Table 5: Telecommands (as defined in `mib.txt`)

TC	ID	APID	Arguments
SET-OBT	1	OBC	hh:mm:ss
REQ-HK	2	OBC	none
SET-PF-MODE	3	OBC	SAFE, NADIR, TARGET
SET-PL-POWER	4	PL	1, 0
PL-TAKE-IMAGE	5	PL	none
REQ-PL-HK	6	PL	none

4.5 Payload Interface

Table 6: Payload interface. The OBC calls the payload directly; the payload reports back through a callback to `OBC.execute()`.

Message	Direction	Call	Meaning
`TAKE-IMAGE`	OBC to PL	`Payload.execute()`	Start a 5 s image take (only when ON)
`TURN-ON`	OBC to PL	`Payload.execute()`	Switch payload ON
`TURN-OFF`	OBC to PL	`Payload.execute()`	Switch payload OFF (also on SAFE)
`HK`	OBC to PL	`Payload.execute()`	Request payload housekeeping
OBT	OBC to PL	`Payload.sync_time()`	Set payload time after SET-OBT
`PL_ON`	PL to OBC	`OBC.execute()`	Payload is ON; OBC sends EVENT `PL_ON`
`PL_OFF`	PL to OBC	`OBC.execute()`	Payload is OFF; OBC sends EVENT `PL_OFF`
`IMG_START`	PL to OBC	`OBC.execute()`	Image take started
`IMG_END`	PL to OBC	`OBC.execute()`	Image taken and logged; OBC sends EVENT `IMG_END`
`HK_ACQ`	PL to OBC	`OBC.execute()`	Payload HK (mode, images, last image); OBC sends HK-PL

5 Modes and onboard time

The OBC boots in SAFE and only accepts payload commands once it has reached NADIR or TARGET; the UML state machine in Figure 2 defines the behaviour of the Mode manager block.

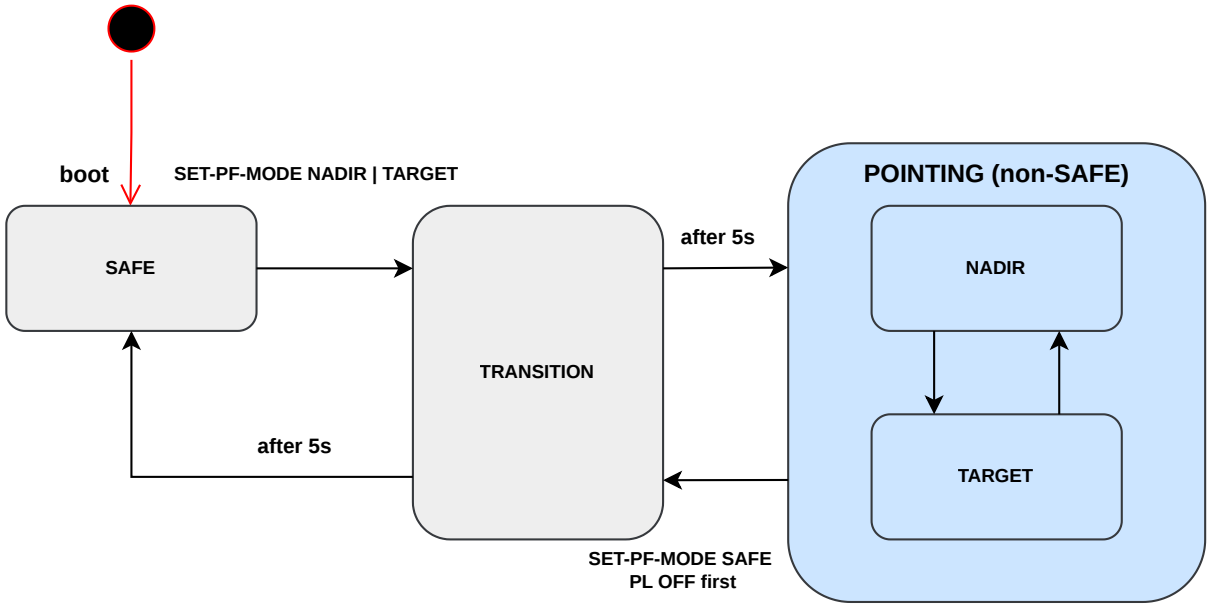


Figure 2: UML state machine of the Platform OBC mode manager. Every `SET-OBC-MODE` command passes through `TRANSITION` for 5 s before the new mode is reached. Commanding SAFE switches the payload off before the tranistion starts.

Every mode change goes through TRANSITION for 5 s; on arrival the OBC sends EVENT

MODE_ACQ with the OBT. During TRANSITION every TC is rejected with NACK 'IN TRANSITION'. When SAFE is commanded, the payload is switched off immediately, before the transition starts.

5.1 Payload states

Table 7: Payload state transitions. Guards are checked by `OBC.check_mode` before the TC is ACKed; a failed guard gives a NACK and no state change.

From	Trigger	Guard	To	Result
–	OBC boot	–	OFF	Payload starts OFF
OFF	SET-PL-POWER 1	OBC in NADIR or TARGET	ON	ACK, EVENT `PL_ON`
ON	PL-TAKE-IMAGE	OBC in NADIR or TARGET	IMAGING	ACK; 5 s timer starts, `IMG_START` reported to OBC
IMAGING	5 s timer expires	–	ON	Image log entry written with OBT, EVENT `IMG_END`
ON, IMAGING	SET-PL-POWER 0	OBC in NADIR or TARGET	OFF	ACK, EVENT `PL_OFF`
ON, IMAGING	SET-OBC-MODE SAFE	–	OFF	Payload switched off before the transition starts, EVENT `PL_OFF`
ON, IMAGING	REQ-PL-HK	OBC in NADIR or TARGET	(same)	ACK, HK-PL (mode, images, last image)
Rejected commands (no state change)				
any	any PL TC	OBC in SAFE	(same)	NACK "OBC IN SAFE MODE"
any	any TC	OBC in TRANSITION	(same)	NACK "IN TRANSITION"
OFF	PL-TAKE-IMAGE, REQ-PL-HK	–	OFF	NACK "PL IS OFF"
OFF	SET-PL-POWER 0	–	OFF	NACK "PL POWER IS AT REQUESTED LEVEL"
ON, IMAGING	SET-PL-POWER 1	–	(same)	NACK "PL POWER IS AT REQUESTED LEVEL"
IMAGING	PL-TAKE-IMAGE	–	IMAGING	NACK, "PL is busy"

5.2 Onboard time

Table 8: Onboard time states (Onboard time block)

State	Reported time	Leaves on	Effect
SCET	Seconds since boot as hh:mm:ss, from 00:00:00	SET-OBT hh:mm:ss	Time-tagged TCs rejected with NACK "OBT not set"
OBT	OBT set by Ground plus elapsed time, hh:mm:ss	–	TIME-SYNC to the payload, event TM "OBT SET"

6 Cyclic execution and key sequences

The OBC and the payload each run a loop every 100 ms; nothing blocks, so a slow link never stops the clock, the scheduler or the mode timers.

6.1 OBC main loop (one cycle, PF8)

1. Wait up to 0.1 s for socket events: a new Ground connection, or incoming TC packets.
2. Handle each complete TC.
3. Check whether a running transition has finished.

6.2 Payload

The payload has no loop of its own. It is driven by calls from the OBC; the 5 s image take runs on a timer.

6.3 TC acceptance (PF3)

Checked in this order; the first failure sends a NACK with its reason.

1. Split the data field.
2. Look up the ID; unknown -> NACK “Unknown or invalid TC ID”.
3. Sequence count; a gap → `EVENT SEQ_GAP`.
4. Mode checks -> NACK with the reason from `check_mode`.
5. ACK, then execute.

6.4 Image take, end to end

1. Ground sends `PL-TAKE-IMAGE`; the OBC sends an ACK.
2. The OBC calls Payload.execute("TAKE-IMAGE"); the payload enters IMAGING and reports `IMG_START`.
3. After 5 s the payload writes the log entry with the OBT, returns to ON and reports `IMG_END`.
4. The OBC sends EVENT `IMG_END` to Ground.

6.5 Switch to SAFE

1. Ground sends `SET-OBC-MODE` SAFE; the OBC sends an ACK.
2. If the payload is on, the OBC turns it off and sends `EVENT PL_OFF`.
3. The OBC enters TRANSITION, and after 5 s sends `EVENT MODE_ACQ` “SAFE”.

7 Traceability matrix

Table 9: Requirements traceability matrix

Req	Requirement	Block(s)	FT functions	Mode transition
R01	Three separate SW programs	All	PF, PL, G	–
R02	Text prompt for TCs	Ground	G2.1, G2.2	–
R03	TM displayed as received	Ground	G3.1, G3.6	–
R04	HK reception timestamp + logfile	Ground	G1.2, G3.5, G3.7	–
R05	MIB as text file	Ground	G1.1, G2.3	–
R06	SAFE: PL cmds rejected, PL OFF	Mode manager, TC handler	PF1.2, PF3.5, PF5.1	SAFE + PL TC → NACK
R07	At least one non-SAFE mode	Mode manager	PF5.1, PF5.2	SAFE → slew → NADIR / TARGET
R08	PL commandable only in non-SAFE	TC handler, Payload interface	PF3.5, PF6.2	PL OFF → ON (guard)
R09	SAFE implies PL shutdown	Mode manager	PF5.4	NADIR / TARGET → SAFE
R10	Mode transitions take time	Mode manager	PF5.2, PF5.3	Transition (10 s / 5 s)
R11	Mode acquisition event TM	Mode manager, TM generator	PF5.3	Transition → mode
R12	Invalid commands rejected with reply	TC handler, PL modes	PF3.2, PF3.4, PF3.7, PF5.5, PL3.2	All NACK rows
R13	Accepted commands acknowledged	TC handler	PF3.7, G3.4	–
R14	Time-tagged TCs	TC handler, Scheduler	PF3.8, PF4.1	–
R15	Overdue time tag rejected	TC handler	PF3.6	–
R16	Time-tagged TC executed when due	Scheduler	PF4.2, PF4.3, PF4.4	–
R17	Common time base Gnd / PF	Onboard time	PF2.2	SCET → OBT
R18	PL OFF / ON simulated	Payload interface, PL modes	PF6.1, PL1.1, PL3.3	PL OFF ↔ ON
R19	PL reports image start and finish	Imaging	PL4.1, PL4.3, PF6.3	PL ON → IMAGING → ON
R20	PL image info to Gnd in HK-PL	Payload interface, TM generator	PF6.5, PL5.1	–
R21	PL ON/OFF event TM	Payload interface	PF6.6	PL OFF ↔ ON
R22	Command target identification	TC handler, Payload interface	PF3.8, PF6.2	–
R23	TC and TM sequence counts	TC handler, TM generator	PF3.3, PF7.4	–
R24	Cyclic HK vs. event TM	TM generator	PF7.1, PF7.2, PF7.3	–
R25	SCET until OBT set	Onboard time	PF1.1, PF2.1	SCET state
R26	SET-OBT TC	Onboard time	PF2.2	SCET → OBT
R27	hh:mm:ss in TM and time tags	Onboard time	PF2.1	–

continued on next page

Req	Requirement	Block(s)	FT functions	Mode transition
R28	OBT periodically in HK	TM generator	PF7.1	–
R29	Event TM carries OBT	TM generator	PF7.3, PF4.4, PF5.3, PF6.6	–
R30	PL image log with take time	Imaging	PL1.3, PL4.2	–
R31	PL knows actual OBT	Onboard time, PL time	PF2.3, PL2.1	SCET → OBT (sync PL)
R32	Cyclic loops, PF polls uplink	Main loops, Ground link	PF8, PF3.1, PL6	–